DARKPOOL

A simplified hidden-role business game with a scalable server architecture

DESIGN AND BUILD PLAN · DRAFT 3

October 2026

Gameplay Simple trading, confidential information, investigation, and voting.	**Scale target** Up to 64 participants per match, with 4 concurrent matches as the first load-test target.
Implementation C++20 on Ubuntu; separate client processes; authoritative server.	**Architecture** Event-driven networking, match isolation, bounded resources, and measurable load tests.

The game rules stay intentionally small. Scalability is handled by the server architecture, not by adding complicated gameplay. This plan separates what must work in the first playable build from the larger capacity targets that must be demonstrated by tests.

This is a proposed design and implementation plan, not evidence that the software already exists or meets the stated capacity targets.

Contents and scope

• 1. Summary and design principles

• 2. Game rules and simplified scope

• 3. Capacity targets and what they mean

• 4. Scalable architecture

• 5. Connection, match, and state-management flow

• 6. Protocol and resource limits

• 7. Security and isolation model

• 8. Repository layout

• 9. Build roadmap

• 10. Load and reliability test plan

• 11. Acceptance criteria

• 12. Risks and decisions

• Appendix A. Example match

• Appendix B. Glossary

Important distinction

Five players is the initial development test, not the scalability ceiling. The architecture and interfaces should not assume exactly five players. The first capacity target is 64 participants in one match and four matches active at the same time on a single server process. These are engineering targets to validate, not pre-existing performance claims.

Design principles

- **Keep gameplay simple:** one Insider, one Investigator, Employees, one company share, and one decision per round.
- **Keep the server authoritative:** clients submit requests; the server validates and applies all game-state changes.
- **Separate connections from matches:** a socket is not the same thing as a player identity or a match.
- **Isolate matches:** messages, roles, trades, clues, votes, timers, and logs must never leak across match IDs.
- **Bound resource use:** limit message size, per-client request rate, queued output, active matches, and total connections.
- **Scale by measurement:** add complexity only when repeatable load tests show a bottleneck.

1. Summary and design principles

DARKPOOL is a business-themed social-deduction game. Players discuss a company's condition, make a simple share decision, use limited private information, and vote on a suspected Insider. It borrows hidden roles and timed discussion from Werewolves, but adds confidential business information and a small financial outcome.

The design should support both a small classroom demonstration and larger sessions without rewriting game logic. The first milestone may run locally with five active players. The server should nevertheless use player IDs, match IDs, bounded queues, and a transport-independent protocol from the beginning.

2. Game rules and simplified scope

2.1 Roles

Role	Objective	Ability
Insider	Damage company performance and avoid being identified.	Receives one confidential event hint; can use the same trading action as other players.
Investigator	Help the Employee team identify the Insider.	Can request one limited clue per match, subject to the rules.
Employee	Protect the company and identify the Insider.	Discuss, trade, and vote.

2.2 Round sequence

- The server announces the round and publishes a business event.
- Players discuss in public chat and may send authorized private messages.
- Each player submits one BUY, HOLD, or SELL decision. The server validates and records it.
- The server resolves the simple market effect and updates cash, shares, company health, and price.
- Players discuss evidence and vote. The server applies the voting rule and records the result.
- After three rounds, the server evaluates the win condition and emits a final summary.

2.3 What remains out of scope for the first release

- Real money, external market feeds, exchange integration, short selling, derivatives, and complex auctions.
- Graphical or mobile clients, public matchmaking, and persistent player accounts.
- Multi-region deployment, distributed consensus, and automatic failover.
- Advanced anti-cheat claims that cannot be tested in the project environment.

3. Capacity targets and what they mean

The document distinguishes a development baseline, a capacity target, and future expansion. Passing a target must be demonstrated by a recorded test run on stated hardware and software.

Level	Target	Purpose	Status
Development baseline	5 active players + moderator	Build and debug the full game loop.	Required first
Normal session	8–16 players per match	Expected small-group usage.	Required before demo sign-off
Single-match capacity	64 player connections in one match	Validate fan-out, timers, voting, state isolation, and queue limits.	Design target; must be tested
Concurrent matches	4 active matches, up to 64 players each where test resources permit	Validate match isolation and shared server resource limits.	Stretch capacity target; report actual result
Beyond initial target	More matches or more than 64 players per match	Only after profiling and a new acceptance target.	Not promised in Draft 3

Interpretation

- **Per-match capacity** means the maximum players participating in one game instance.
- **Concurrent-match capacity** means multiple independent games running in the same server process.
- **Total connections** can exceed the number in one match. For example, four full 64-player matches imply 256 player connections, plus moderators and test clients.
- If the development machine cannot test 256 simultaneous clients, record the tested maximum and environment; do not report the target as achieved.

Capacity is not a guess about how many sockets Linux can open. It is an end-to-end property of message fan-out, application logic, output queues, memory, CPU, timers, and failure handling.

Initial non-functional objectives

Metric	Proposed acceptance target	Measurement notes
Command handling latency	p95 ≤ 100 ms at 64 connected clients under the defined test workload	Measure client send-to-server acknowledgement; exclude deliberate game-phase waiting.
Server stability	No crash, deadlock, or unbounded memory growth during a 30-minute steady-load run	Record RSS, CPU, queue depths, and process exit status.
Queue behavior	Queues remain bounded; slow clients are throttled or disconnected by policy	Force one client to stop reading and verify other clients remain responsive.
Isolation	Zero cross-match message, role, clue, trade, or vote leaks	Run simultaneous matches with unique canary strings and compare received logs.
Protocol robustness	Malformed, oversized, unauthorized, and over-rate messages are rejected without server failure	Automated negative tests and fuzzing where feasible.

These latency numbers are proposed project acceptance thresholds, not measured results. If the target cannot be met on the documented test machine, report the result and bottleneck rather than silently changing the result.

4. Scalable architecture

Use a single authoritative server process for the first scalable version. The process owns all game-state mutations. It accepts connections, authenticates sessions, routes requests by match ID, and maintains independent state for each active match. This avoids premature distributed-systems complexity while supporting many players and concurrent matches.

4.1 Logical components

Component	Responsibility	Scaling / safety rule
Transport adapter	Accept local UNIX-socket connections initially; later add TCP with TLS 1.3 through the same interface.	Transport code must not implement game rules.
Connection manager	Track sockets, framing, connection state, output queues, rate limits, and disconnects.	Nonblocking I/O; bounded per-client queues; no thread per client.
Authentication/session manager	Map an authenticated connection to a stable player identity and permissions.	Never trust a client-supplied player ID as proof of identity.
Match manager	Create, find, close, and route to matches.	Every request must be authorized against match membership.
Match state machine	Own roles, phase, deadlines, company state, trades, clues, votes, and win conditions.	State mutations occur on the server's owner/event loop.
Event/audit log	Record important state transitions and security-relevant events.	Do not log secret role details into player-visible logs.
Worker pool (optional)	Handle future CPU-heavy tasks, such as expensive analytics.	Workers return results to the owner loop; they do not mutate match state directly.

4.2 Event-driven I/O

- Use nonblocking sockets with `epoll` on Linux. One event loop can manage many mostly idle client connections without one operating-system thread per client.
- Use monotonic-clock deadlines for phase timers. `timerfd` is a practical Linux option; a deadline heap checked by the event loop is also acceptable.
- Keep request processing short. Parse, authenticate, validate, apply a bounded state transition, and queue responses.
- Never block the event loop on a slow client, file write, DNS lookup, or expensive computation.
- Start with one state-owning event loop. Add worker threads only for measured CPU-heavy work, and marshal their results back to the owner loop.

4.3 Match isolation

Each match should be represented by a Match object keyed by an unguessable or non-reused match identifier. It owns its player membership, role assignments, game phase, deadlines, company state, trades, chat routing, votes, and match log. A request is accepted only if the authenticated session belongs to the target match and the action is legal in the current phase.

A match should not hold direct pointers to client-owned buffers or assume clients stay connected. Disconnect/reconnect policy, timeouts, and stale session cleanup must be explicit. One match timing out or producing many messages must not stall all other matches.

4.4 Architecture sketch

Clients (local UNIX socket now; TCP/TLS later) → nonblocking transport and framing → session/authentication → match router → independent Match state machines → bounded outbound queues → clients

5. Connection, match, and state-management flow

5.1 Connection lifecycle

1. Accept a connection and assign a server-side connection ID.
2. Authenticate the user/session using the selected transport's trust mechanism.
3. Create or resume a player session according to the policy.
4. Join a match through a server-validated join operation.
5. Route all game actions using the server-side session-to-match mapping.
6. On disconnect, mark the session disconnected, apply a bounded reconnect grace period if enabled, and clean up stale resources.

5.2 Request validation pipeline

For every request: validate frame length → parse message type → validate fields and limits → identify authenticated session → check match membership → check phase and role permission → check rate limit → apply state transition → append audit event → enqueue authorized response(s).

5.3 Message envelope

Use a versioned, length-prefixed message envelope. A small binary protocol is suitable for C++, but a clearly specified newline-delimited JSON protocol can be easier to debug in the first prototype. Whichever is chosen, keep parsing separate from game logic and do not deserialize arbitrary object graphs.

Illustrative fields (not a complete wire specification):

```
version, message_type, request_id, match_id, payload_length, payload
```

Minimum protocol rules

- Set a hard maximum frame size (proposed initial limit: 4 KiB for ordinary commands; larger server events must be explicitly bounded).
- Reject invalid versions, unknown required message types, truncated frames, invalid UTF-8 where text is expected, and malformed fields.
- Set maximum chat length, whisper length, pending output bytes, and requests per time window.
- Do not allow clients to submit authoritative cash, share counts, roles, company health, votes for other users, or final outcomes.
- Use request IDs for correlation and safe duplicate handling; define whether retried actions are idempotent.
- Apply explicit backpressure: pause reads, reject nonessential messages, or disconnect a persistently slow consumer according to a documented policy.

6. Security and isolation model

The server is the trust boundary. The goal is to make cheating and accidental data leaks difficult, and to make security claims testable. A local socket or a separate Linux account helps but does not automatically make the entire game secure.

Threat	Required control	Test evidence
Forged player identity	Bind identity to authenticated session; ignore claimed sender IDs.	Attempt to send as another player; server rejects it.
Cross-match leakage	Route by authenticated membership; separate role and chat visibility.	Canaries and secret-role checks across two simultaneous matches.
Illegal action / phase abuse	Central validation in match state machine.	Submit trade, clue, or vote at wrong phase and verify no state change.
Flooding / oversized input	Frame limits, rate limits, bounded queues, per-session quotas.	Flood test; verify rejection and continued service for other clients.
Slow consumer	Bounded outbound queue and backpressure/disconnect policy.	Pause reads from one client; verify server memory and other-client latency remain bounded.
Local account/process abuse	Least privilege, restrictive socket permissions, separate service account where practical.	Verify unauthorized local user cannot connect or read protected files.
Secret leakage in logs	Separate moderator/audit logs from player-visible output.	Search player output and logs for role and clue canaries.
Network interception (future TCP)	TLS 1.3 and certificate-based client authentication or another explicit identity scheme.	Verify invalid certificate and plaintext connection are rejected.

Security implementation order

- First: server authority, validation, message bounds, match authorization, least privilege, and safe cleanup.
- Second: automated negative tests, fuzzing of the parser, resource limits, and audit log review.
- Third: TCP/TLS transport and certificate lifecycle, if remote networking is required.
- Later: seccomp, systemd sandboxing, Landlock, cover traffic, or advanced side-channel mitigation only when the project has a clear testable requirement.

Do not claim resistance to a compromised host, a malicious root user, or all timing side channels. State the threat model and test results precisely.

7. Repository layout

Keep the first implementation modular without splitting it into many services. One daemon can contain distinct modules with clear interfaces.

```
darkpool/
■■■ CMakeLists.txt
■■■ README.md
■■■ docs/
■ ■■■ protocol.md
■ ■■■ threat-model.md
■ ■■■ architecture.md
■ ■■■ test-plan.md
■■■ include/darkpool/
■ ■■■ protocol.hpp
■ ■■■ transport.hpp
■ ■■■ session.hpp
■ ■■■ match.hpp
■ ■■■ match_manager.hpp
■ ■■■ audit_log.hpp
■■■ src/
■ ■■■ server_main.cpp
■ ■■■ client_main.cpp
■ ■■■ transport_unix.cpp
■ ■■■ connection_manager.cpp
■ ■■■ session_manager.cpp
■ ■■■ match.cpp
■ ■■■ match_manager.cpp
■ ■■■ protocol.cpp
■■■ tests/
■ ■■■ unit/
■ ■■■ integration/
■ ■■■ security/
■ ■■■ load/
■■■ scripts/
■■■ run_local_demo.sh
■■■ run_load_test.sh
```

8. Step-by-step build roadmap

Stage	Deliverable	Definition of done	Estimate
S0 — Rules & protocol	Rules, phase diagram, message types, threat model.	Protocol and legal actions documented; no ambiguous authority boundaries.	3–4 h
S1 — Game engine	Match state machine without sockets.	Unit tests cover roles, trades, clues, votes, deadlines, and win conditions.	5–7 h
S2 — Local transport	C++ server/client over UNIX domain sockets.	Five clients complete a full match; disconnects do not crash server.	6–8 h
S3 — Sessions & routing	Player sessions, match manager, multiple matches.	Two matches run simultaneously with automated isolation tests.	5–7 h
S4 — Event loop & limits	Nonblocking epoll loop, bounded queues, timers, rate limits.	Slow-client and flood tests pass without unbounded resource growth.	6–8 h
S5 — Client UX & demo	Terminal client, moderator commands, clear phase messages.	Reproducible demo with public/private messages, trades, votes, and results.	4–6 h
S6 — Test harness	Simulated clients and metrics collection.	5/8/16/32-client runs are reproducible and logged.	4–6 h
S7 — Capacity tests	64-client single match and concurrent-match test attempts.	Report actual capacity, latency, CPU, memory, and failures.	4–7 h
S8 — Security pass	Negative tests, parser fuzzing, permissions, audit review.	All required security tests have recorded pass/fail evidence.	5–8 h
S9 — TCP/TLS (optional)	Remote transport adapter, TLS, client identity.	Valid remote clients connect; invalid certificates and plaintext are rejected.	8–14 h

Estimated local scalable prototype: approximately 42–61 hours before optional TCP/TLS. These are planning estimates and may change based on debugging, test environment, and required evidence.

9. Load and reliability test plan

Load testing should use automated clients that follow the real protocol. A test that only opens sockets does not demonstrate game capacity. Simulated clients should send a realistic mixture of joins, chat, trades, votes, and phase acknowledgements.

9.1 Test matrix

Run	Players / matches	Workload	Required observations
L1	5 players / 1 match	Full game, normal pacing.	Correct behavior; baseline CPU/RSS; no protocol errors.
L2	8 and 16 / 1 match	Chat, trades, votes at realistic rates.	p50/p95 latency; CPU/RSS; event-loop responsiveness.
L3	32 / 1 match	Sustained messaging and phase changes.	Queue depth, dropped/rejected requests, latency, correctness.
L4	64 / 1 match	Sustained load for at least 10 minutes.	No crash; bounded memory/queues; report latency and CPU.
L5	2 matches × 32 players	Concurrent actions in both matches.	No cross-match leak; neither match starves the other.
L6	4 matches × up to 64 players	Stress target; attempt where machine permits.	Record actual connections, RSS, CPU, latency, errors, and any limit reached.
L7	Slow / disconnected clients	One client stops reading; clients disconnect mid-round.	Bounded queue; correct cleanup; other matches remain responsive.
L8	Malformed / flood input	Oversized frames, invalid fields, high-rate chat/actions.	Rejected safely; no crash, memory growth, or state corruption.

9.2 Metrics to capture

- Hardware and environment: CPU model/core count, RAM, Ubuntu version, compiler/build mode, kernel version.
- Capacity: connected clients, active matches, accepted/rejected connections, reconnects and disconnects.
- Latency: command acknowledgement and broadcast delivery p50/p95/p99, where meaningful.
- Resource use: process RSS, CPU utilization, open file descriptors, queue depth, event-loop delay.
- Correctness: illegal actions rejected, valid actions processed, duplicate actions, state invariant failures, cross-match leaks.
- Reliability: crashes, timeouts, disconnect handling, and duration of steady-load run.

9.3 Reproducibility

The load-test script should accept client count, match count, run duration, and workload profile as parameters. Save a timestamped CSV or JSON report plus server logs. Repeat each significant run at least three times where practical. Include the exact command and environment in the final report.

Example command shape (implement to match the eventual harness):

```
./scripts/run_load_test.sh --players 64 --matches 1 --duration 600 --profile mixed
```

Do not present a target as a result. The report must clearly label each row as PASS, FAIL, NOT RUN, or BLOCKED and include evidence.

10. Acceptance criteria

The project is ready for a demo when the required functional and security criteria pass. The scalable capacity claim is ready only after the associated load test passes.

ID	Acceptance criterion	Evidence
F-01	Five active clients complete all three rounds.	Automated integration test and demo transcript.
F-02	Roles are assigned server-side and hidden from unauthorized clients.	Role-visibility tests and moderator log.
F-03	Public chat and permitted private messages route correctly.	Multi-client transcript.
F-04	Trades, clue actions, and votes are validated by phase and permission.	Unit and negative tests.
F-05	Two or more matches run at the same time.	Concurrent integration test with independent outcomes.
S-01	Cross-match messages, roles, clues, trades, and votes never leak.	Canary isolation test.
S-02	Oversized and malformed messages are rejected safely.	Protocol test suite; no crash or state corruption.
S-03	Slow clients cannot create unbounded output queues.	Slow-reader test and queue/RSS measurements.
S-04	Rate limits constrain abusive clients without blocking normal play.	Flood test and recorded thresholds.
C-01	16-player session passes the normal-session test.	Three recorded runs with metrics.
C-02	64-player single match reaches the stated performance threshold.	Three recorded runs; if not met, document measured limit and bottleneck.
C-03	Four concurrent matches are tested; capacity and isolation results are reported.	Load report; target is not claimed without passing evidence.

Demo evidence checklist

- Show server and several independent player clients in separate terminals.
- Show a public message and a private message, including proof the private message is not broadcast to others.
- Show role assignment privately, one clue action, trading, voting, and a completed match.
- Show two match IDs active simultaneously and demonstrate no cross-match leakage.
- Show a load-test summary with measured results, not just a successful compilation.
- Show at least one rejected invalid action and one rejected malformed/oversized request.

11. Risks and decisions

Risk	Why it matters	Mitigation
Trying to implement all capacity goals at once	Debugging game rules, protocol, and performance simultaneously is difficult.	Build in stages; use five players first, then increase client count.
Too much work on threads	Shared mutable match state can create race conditions and nondeterministic votes.	One state owner/event loop first; introduce workers only for measured needs.
Unbounded fan-out queues	A slow player can consume memory and delay unrelated matches.	Per-connection queue caps, backpressure, and a slow-client policy.
One match blocks another	Synchronous logging or expensive work can stall the server.	Nonblocking I/O, bounded per-event work, metrics, and isolation tests.
Overclaiming scalability	A design target is not proof of actual capacity.	Report tested concurrency, machine specs, latency percentiles, and failures.
Transport and game logic become coupled	Adding remote TLS later can require a rewrite.	Define a transport interface and keep the game protocol transport-independent.

Recommended implementation order

Complete a correct five-player match first, but do not hard-code five into the architecture. Add match IDs and concurrent-match isolation early. Then move to nonblocking I/O and resource limits before attempting 32- and 64-client load tests. Add TCP/TLS only after the local protocol and state machine are stable.

Decisions that must be documented during implementation

- Maximum players per match and maximum simultaneous matches configured by the server.
- Whether a disconnected player can reconnect, and for how long.
- Whether a match pauses when its moderator disconnects.
- Queue overflow behavior and rate-limit thresholds.
- Whether chat is persisted and exactly which events are included in audit logs.
- The chosen protocol encoding, frame limit, and version compatibility policy.

Appendix A. Example match

A local demo starts one server and five clients. The server creates match M-104, assigns one Insider, one Investigator, and three Employees, then announces Round 1. Players discuss a product delay in public chat. The Insider receives a private hint that the launch is likely to be delayed; other players do not receive that hint. Players submit BUY, HOLD, or SELL, and the server resolves the event. The Investigator uses the single clue action, the group votes, and the server announces only the result the rules permit.

At the same time, another group plays match M-105. A message in M-104 must never be sent to M-105. The role assignment, clue response, share balances, votes, phase timer, and win condition for each match remain separate. A load test repeats this pattern with many simulated clients.

Appendix B. Glossary

Term	Meaning
Authoritative server	The server decides valid actions and owns the official game state.
Match	One independent instance of the game, with its own players and state.
Match ID	Identifier used to route requests and events to the correct match.
Event loop	A loop that reacts to socket readiness and timers without blocking on each client.
epoll	Linux API for monitoring many file descriptors efficiently.
Backpressure	A policy that slows, rejects, or disconnects producers when consumers cannot keep up.
Bounded queue	A queue with a maximum size, preventing unlimited memory growth.
p95 latency	The latency at or below which 95% of measured operations complete.
IPC	Inter-process communication; here, local clients communicate with the server using sockets.
TLS 1.3	A modern transport security protocol proposed for the later remote-networking milestone.

Final note

Draft 3 keeps the game intentionally simple while making scalability an explicit architectural and testing requirement. The project should claim only the player counts and performance levels demonstrated by repeatable tests.